מבוא למדעי המחשב

הרצאה 8.1: מבנים (STRUCT)

מה נלמד

- מבנים (לעומת מחלקות)
- typedef למבנים
- השמת מבנים
- מבנים ומצביעים
- העברת מבנים לפונקציות
- השוואה בין מבנים
- הכלת מבנים

מבנים

- מבנה (struct) הוא טיפוס חדש המכיל אוסף של שדות בעלי קשר לוגי
- בשונה מ class (C++, Java, etc.) מכיל רק נתונים (ללא פעולות)
- השמה בין מבנים מעתיקה את השדות אחד-אחד
- מערך של מבנים:
  - מכיל את המידע בתוך האיבר במערך בדומה לטיפוסים פשוטים, אינו מערך של מצביעים

מבנה דוגמא

שם הטיפוס החדש

```
struct Date
{
    int day;
    int month;
    int year;
};


void main()
{
    struct Date d;
    d.day = 23;
    d.month = 8;
    d.year = 2008;

    printf("The date is %d/%d/%d\n", d.day, d.month, d.year);
}
```

הגדרת מבנה המכיל אוסף של שדות

הגדרת משתנה מטיפוס המבנה

פניה לשדות המשתנה תהיה באמצעות נקודה

מבנה - הגדרה

- מבנה חדש יוגדר לפני השמוש בו (לרוב בתחילת הקובץ או בקובץ h)
- אופן ההגדרה:

```
struct <שם המבנה>
{
    <type1> <name1>;
    <type2> <name2>;
};
```

- פניה לשדה במבנה על ידי:

```
<שם המשתנה>.<שם השדה>
```

הגדרת טפוס חדש - typedef

- הגדרת טפוס מעבר לטפוס פרימטיבי (int, float, char ...)
- typedef מאפשר לתת שם חדש לטפוס
- תחביר:

typedef <שם קיים> <שם חדש>

- דוגמאות:

typedef unsigned long ulong

כעת נוכל להגדיר משתנה מטיפוס זה באחת משתי הדרכים:

unsigned long id; או ulong id;

typedef int bool

שימוש ב- typedef למבנה

```
#include <stdio.h>

struct  Date
{
      int day;
      int month;
      int year;
};

typedef  struct  Date  date_t;

void main()
{
      date_t  d;
      d.day = 23;
      d.month = 8;
      d.year = 2008;

      printf("The date is %d/%d/%d\n", d.day, d.month, d.year);
}
```

שימוש ב- typedef ישירות למבנים

```
typedef struct
{
    int day;
    int month;
    int year;
} date_t;
```

איתחול מבנה

- ניתן לאתחל מבנה בשורת ההגדרה כמו שמאתחלים מערך:

```
typedef struct
{
   int day, month, year;
} date_t;

void main()
{
     date_t  d= {23, 8, 2003};
     printf("The date is %d/%d/%d\n", d.day, d.month,d.year);
}
```

- הערך הראשון יושם בשדה הראשון, הערך השני בשדה השני וכו'
- יש לוודא התאמה בין טיפוס השדה לערך
- השמת ערכים שלא בשורת ההגדרה ניתן לבצע רק שדה-שדה ע"י השמה

השמת מבנים

- השמה בין מבנים מעתיקה שדה-שדה
- שדה שהוא מערך, מועתק באופן אוטומטי איבר-איבר
  - בניגוד להשמה בין מערכים שאינם בתוך מבנה (צריך בלולאה להעתיק איבר-איבר)

```
#include <stdio.h>

struct Student
{
	char  name[6];
	float  age;
	long  id;
} typedef  student_t;

void main()
{
	student_t  s1 = {"momo", 23.5, 111111};
	student_t  s2;
	s2 = s1;
}
```

שימו לב – גישה לשדה – כמו טיפוס המשתנה

- למשל, אם היינו רוצים לבצע השמה לאחד משדות המבנה שהוא מחרוזת, היינו צריכים להשתמש ב- strcpy

```
struct  Student
{
    char  name[6];
    float  age;
    long   id;
} typedef  student_t;
```

```
void main()
{
    student_t  s1;

    strcpy(s1.name, "momo");
    s1.age = 23.5;
    s1.id = 111111;

    printf("The second student's details:\nname=%s, age=%.2f,
    id=%ld\n", s1.name, s1.age, s1.id);
}
```

העברת מבנה לפונקציה

- כאשר מעבירים מבנה לפונקציה, מועבר העתק שלו, בדיוק כמו העברת פרמטר ממשתנה פרימיטיבי
  - המשתנה מועבר **by value**
  - לכן - אם נשנה את אחד משדות המבנה בפונקציה, השינוי לא ישפיע על המשתנה המקורי

```
typedef struct
{
    int  day, month, year;
} date_t;

void printDate(date_t  d,
               char delimiter)
{
    printf("%d%c%d%c%d",
    d.day, delimiter, d.month,
    delimiter, d.year);
}

void main()
{
    date_t  d1 = {31, 8, 2007};

    printDate(d1, '/');
    printf("\n");
    printDate(d1, '-');
    printf("\n");
}
```

אופרטורי יחס בין מבנים

- בניגוד למשתנה פרימיטיבי, לא ניתן להשוות בין מבנים ע"י אופרטורי ההשוואה: ==, =!, >, < , =>, =<
- סיבות:
  1. יתכן ואחד משדות המבנה אינו ניתן להשוואה ע"י אופרטורים אלו, למשל מערך או מחרוזת
  2. אם ישנם כמה שדות במבנה, לא ברור לפי מה נחליט לאיזה שדה חשיבות גדולה יותר (למשל השוואה בין סטודנטים: לפי ת.ז. או לפי שם?), וכמובן הקומפיילר אינו יכול לנחש זאת
- כדי להשוות בין מבנים עלינו לכתוב פונקציות מתאימות
- גם לביצוע פעולות חשבון בין שני מבנים עלינו לכתוב פונקציות (האם מוגדרת פעולת החיבור בין 2 סטודנטים? ועבור שני מערכים?)

השוואה ויחס בין מבנים

- כל אופרטורי היחס (> < == ועוד) לא פועלים על מבנים. יש לכתוב פונקציה יעודית
- פונקצית השוואה לדוגמא:

```
struct  Clock
{
    int  hour, minute;
} typedef  clock_t;

int  equalClock(clock_t  c1, clock_t  c2)
{
    return (c1.hour == c2.hour &&
            c1.minute == c2.minute);
}
```

מבנה בתוך מבנה

- ניתן להגדיר מבנה כשדה במבנה אחר:
  - דוגמא:

```
struct  Date
{
    int  day, month, year;
} typedef  date_t;

struct Student
{
    char     name[20];
    date_t  birthday;
    float     average;
} typedef  student_t;
```

מערך של מבנים

- אוסף הסטודנטים הרשומים בכיתה

```
#define MAX_STUDENTS 3

struct  Student
{
      char  name[20];
      float  average;
} typedef  student_t;


void main()
{
      student_t students[MAX_STUDENTS] = { {"momo", 90.5},
                                          {"yoyo", 85},
                                          {"gogo", 78.6} };

      printf("The second student is: %s\n", students[1].name);
}
```

אתחול המערך

הסטודנט השני במערך